gemm_systolic_array

Systolic array Output-Stationary P×P

Port list, module con, vi kiến trúc và timing chính xác theo chu kỳ

Bộ tăng tốc GEMM INT8 trên Chipyard SoC

Module	`gemm_systolic_array` (instance `u_array` trong `gemm_accel_top`)
Module con	`gemm_tile_seq`, `gemm_skew`, `gemm_pe_grid`, `gemm_pe`, `gemm_drain`
Tuân theo	Spec hệ thống v1.0: mục 2.1 (FUN-01, FUN-02, FUN-07), mục 4.3 (quyền truy cập SRAM), mục 5.5 (ARR-01 … ARR-08)
Kèm theo	`gemm_systolic_array_model.py`: mô hình tham chiếu cycle-accurate của đúng vi kiến trúc này; kết quả và mọi thời điểm trong tài liệu được kiểm bằng mô hình đó

Mục lục

1 Chức năng **3**
1.1 Thuật toán . . . 3

2 Tham số **3**

3 Port list **4**
3.1 Quy tắc giao tiếp . . . 4

4 Vi kiến trúc **5**

5 Module con **6**
5.1 `gemm_pe` . . . 6
5.2 `gemm_tile_seq` . . . 7
5.3 `gemm_skew` . . . 8
5.4 `gemm_pe_grid` . . . 8
5.5 `gemm_drain` . . . 8
5.6 `o_busy` và `o_pe_done` . . . 9

6 Timing **9**
6.1 Dạng sóng minh họa . . . 10

7 Reset và soft clear **10**

8 Quy tắc hiện thực **11**

1 Chức năng

Sau một xung `i_pe_start`, array tự duyệt mọi tile P×P của C, tự đọc A và B từ scratchpad, tính tích lũy INT32 trong các PE, rồi xả kết quả sang VPU theo từng hàng, mỗi chu kỳ một hàng. Khi hàng cuối cùng đã đi qua VPU và được ghi vào buffer C, array phát một xung `o_pe_done`. Array không bao giờ chờ ai và không có đầu ra lỗi.

Bảng 1.1: Chức năng

ID	Yêu cầu
SA-01	Với cấu hình M, N, K của `i_matrix_dim`, array xuất đúng một lần mỗi phần tử $C[i][j] = \sum_k A[i][k] \cdot B[k][j]$, 0 ≤ i < M, 0 ≤ j < N, ở dạng INT32 (tích 16 bit có dấu, cộng dồn wrap mod 2^{32})
SA-02	Kết quả được xuất theo đoạn hàng: một lần xuất gồm hàng i, cột $j_0 \ldots j_0 + P - 1$ và mask P bit; lane có mask = 0 mang giá trị không xác định
SA-03	Kết quả không phụ thuộc P và không phụ thuộc `L_VPU`
SA-04	Array chỉ đọc buffer A, B và chỉ xuất sang VPU; không đọc buffer C, không truy cập DRAM, không truy cập buffer BIAS

1.1 Thuật toán

C được chia thành lưới $T_M \times T_N$ tile, $T_M = \lceil M/P \rceil$, $T_N = \lceil N/P \rceil$; tile thứ n có (t_i, t_j) với $n = t_i T_N + t_j$, phủ hàng $i_0 = t_i P \ldots i_0 + P - 1$ và cột $j_0 = t_j P \ldots j_0 + P - 1$. Trong tile, PE(r, c) tích lũy $C[i_0 + r][j_0 + c]$ qua toàn bộ k = 0 … K−1 (Output-Stationary). Chi tiết lý thuyết ở tài liệu “Thuật toán GEMM, systolic array và VPU”, mục 5.

2 Tham số

Bảng 2.1: Tham số

Tham số	Mặc định	Ý nghĩa
`P`	8	Kích thước array; chỉ nhận 4 hoặc 8. Nối từ `PE_ARRAY_SIZE` của top
`L_VPU`	0	Độ trễ của VPU (chu kỳ), 0 … 7; phải bằng đúng độ trễ của `gemm_vpu` được instance
`IDX_W`	5	localparam: độ rộng chỉ số hàng/cột/k (0 … 31)
`DIM_W`	6	localparam: độ rộng giá trị M, N, K (1 … 32)

Bảng 2.2: Tham số

ID	Yêu cầu
SA-05	P ∉ {4, 8} hoặc `L_VPU` ∉ [0, 7] phải bị chặn lúc elaboration (`$error` trong `generate`)

3 Port list

Bảng 3.1: Port của `gemm_systolic_array`

Port	Rộng	Chiều	Nối với	Mô tả
Clock, reset				
`clk`	1	In	top	Clock
`rst_n`	1	In	`rst_n_sync`	Reset bất đồng bộ tích cực thấp, đã đồng bộ sườn nhả
`i_soft_clr`	1	In	FSM	Xung xóa đồng bộ 1 chu kỳ (`soft_clr`)
Điều khiển và cấu hình				
`i_pe_start`	1	In	FSM	Xung 1 chu kỳ: bắt đầu tính toàn bộ tác vụ
`o_pe_done`	1	Out	FSM	Xung 1 chu kỳ: mọi kết quả đã được ghi vào buffer C
`o_busy`	1	Out	FSM	Mức: array đang làm việc
`i_matrix_dim`	32	In	FSM	Kích thước của job đang tính (`o_comp_dim` của FSM): [7:0] M, [15:8] N, [23:16] K
Cổng đọc buffer A (array là master, không có ready)				
`o_a_rd_en`	1	Out	SRAM	Yêu cầu đọc một vector A
`o_a_rd_row`	5	Out	SRAM	i_0 (bội của P)
`o_a_rd_k`	5	Out	SRAM	k
`i_a_rd_data`	8P	In	SRAM	Lane r (bit [8r+7 : 8r]) = $A[i_0 + r][k]$; hợp lệ 1 chu kỳ sau yêu cầu
Cổng đọc buffer B				
`o_b_rd_en`	1	Out	SRAM	Yêu cầu đọc một vector B
`o_b_rd_k`	5	Out	SRAM	k
`o_b_rd_col`	5	Out	SRAM	j_0 (bội của P)
`i_b_rd_data`	8P	In	SRAM	Lane c (bit [8c+7 : 8c]) = $B[k][j_0 + c]$; hợp lệ 1 chu kỳ sau yêu cầu
Luồng hàng kết quả sang VPU (không có ready)				
`o_row_valid`	1	Out	VPU	Có một đoạn hàng kết quả trong chu kỳ này
`o_row_idx`	5	Out	VPU	Hàng i
`o_row_col`	5	Out	VPU	Cột đầu j_0 (bội của P)
`o_row_data`	32P	Out	VPU	Lane c (bit [32c+31 : 32c]) = $C[i][j_0 + c]$, INT32
`o_row_mask`	P	Out	VPU	Bit c = 1: lane c hợp lệ và phải được ghi

3.1 Quy tắc giao tiếp

FSM ↔ Array

TRAO ĐỔI GÌ?

`i_pe_start` bắt đầu một tác vụ; `o_pe_done` báo xong; `o_busy` cho biết đang chạy.

AI KHỞI TẠO?

FSM phát `i_pe_start`; array tự chạy tới cuối và phát `o_pe_done`.

KHI NÀO ĐƯỢC CHẤP NHẬN?

`i_pe_start` chỉ được phát khi `o_busy` = 0. Từ `i_pe_start` tới `o_pe_done`, `i_matrix_dim` phải giữ ổn định.

KHI NÀO SANG BƯỚC TIẾP?

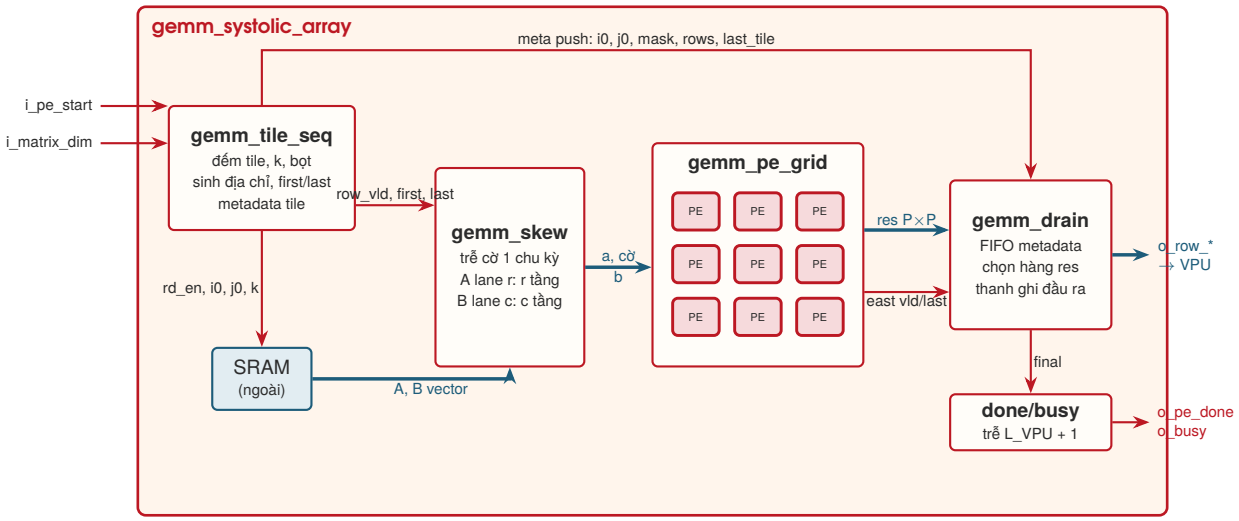
Sau `o_pe_done`, FSM sang STORE_C; array về rảnh ở chu kỳ kế tiếp.

Bảng 3.2: Giao tiếp

ID	Yêu cầu
SA-06	Chu kỳ có `i_pe_start` = 1 gọi là chu kỳ 0. Array chốt M, N, K ở cạnh clock cuối chu kỳ 0. `o_busy` = 1 từ chu kỳ 1 đến hết chu kỳ có `o_pe_done` = 1
SA-07	Mỗi `i_pe_start` được chấp nhận có đúng một `o_pe_done`. `i_pe_start` khi `o_busy` = 1 bị bỏ qua (và vi phạm assertion AS-01)
SA-08	Giá trị M, N, K được giả định hợp lệ (FSM đã kiểm tra): $1 \le M \le 32$; N, K $\in$ {4, 8, …, 32}. Với giá trị khác, hành vi không xác định
SA-09	`o_a_rd_en` và `o_b_rd_en` luôn bằng nhau. Khi bằng 1: `o_a_rd_row` và `o_b_rd_col` là bội của P; `o_a_rd_k` = `o_b_rd_k` < K. Khi bằng 0, các port địa chỉ có giá trị tùy ý
SA-10	SRAM trả dữ liệu đúng 1 chu kỳ sau chu kỳ có yêu cầu; array không dùng `i_a_rd_data`, `i_b_rd_data` ở chu kỳ không có yêu cầu trước đó. Lane ứng với hàng $\ge$ M hoặc cột $\ge$ N được phép mang giá trị bất kỳ, kể cả X trong mô phỏng
SA-11	Mọi port `o_row_*` đều lấy từ thanh ghi. Khi `o_row_valid` = 1: `o_row_idx` < M; `o_row_mask[c]` = 1 khi và chỉ khi $j_0 + c < N$. VPU phải nhận đoạn hàng ở đúng chu kỳ đó
SA-12	Không có hai đoạn hàng cho cùng một (i, j_0) trong một tác vụ; mọi (i, j_0) với i < M, j_0 < N đều xuất hiện
SA-13	`o_pe_done` = 1 đúng `L_VPU` + 1 chu kỳ sau chu kỳ có `o_row_valid` = 1 của đoạn hàng cuối cùng. Với `L_VPU` = 0, VPU là đường thẳng và lệnh ghi C có hiệu lực ở cạnh clock cuối chu kỳ `o_row_valid`; do đó tại lúc `o_pe_done`, mọi kết quả đã nằm trong buffer C

4 Vi kiến trúc

Array gồm 4 khối: **bộ tuần tự tile** phát yêu cầu đọc và cờ điều khiển; **khối làm lệch** trễ lane r của A r chu kỳ và lane c của B c chu kỳ; **lưới PE** P×P; **khối xả** lấy kết quả từng hàng khi hàng đó vừa xong và xuất sang VPU. Mọi khối chạy theo nhịp cố định, không có bắt tay bên trong.



Hình 4.1: Cấu trúc bên trong `gemm_systolic_array` (lưới vẽ 3×3 tượng trưng cho P×P)

Bảng 4.1: Module con

Module	Instance	Vai trò
`gemm_tile_seq`	`u_seq`	Chốt M, N, K; đếm tile và k; phát yêu cầu đọc A, B; sinh cờ `row_vld`, `first`, `last`; đẩy metadata tile cho khối xả
`gemm_skew`	`u_skew`	Trễ cờ điều khiển 1 chu kỳ cho khớp độ trễ SRAM; làm lệch A (cùng cờ) và B

Bảng 4.1: Module con (tiếp theo)

Module	Instance	Vai trò
`gemm_pe_grid`	`u_grid`	P×P instance `gemm_pe` nối thành lưới
`gemm_pe`	`u_pe[r][c]`	Một phần tử xử lý: nhân-cộng, giữ `acc`, `res`
`gemm_drain`	`u_drain`	FIFO metadata; chọn hàng `res` đang sẵn sàng; thanh ghi đầu ra `o_row_*`; cờ `final`
(logic trong top)	—	`o_busy`; đường trễ `L_VPU` + 1 từ `final` tới `o_pe_done`

5 Module con

5.1 gemm_pe

Bảng 5.1: Port của `gemm_pe`

Port	Rộng	Chiều	Mô tả
`clk`, `rst_n`	1, 1	In	Clock, reset
`i_clr`	1	In	Xóa đồng bộ (nối `i_soft_clr`)
`i_a`	8	In	A, INT8 có dấu, từ PE bên trái (hoặc khối làm lệch)
`i_vld`	1	In	Cặp (a, b) của chu kỳ này hợp lệ
`i_first`	1	In	Phần tử k = 0 của tile
`i_last`	1	In	Phần tử k = K−1 của tile
`i_b`	8	In	B, INT8 có dấu, từ PE bên trên (hoặc khối làm lệch)
`o_a`, `o_vld`, `o_first`, `o_last`	8, 1, 1, 1	Out	Bản trễ 1 chu kỳ của đầu vào tương ứng, sang PE bên phải
`o_b`	8	Out	Bản trễ 1 chu kỳ của `i_b`, xuống PE bên dưới
`o_res`	32	Out	Thanh ghi kết quả `res`: tổng của tile gần nhất đã xong

$$sum = (i_first\ ?\ 0 : acc) + \mathrm{sext}_{32}(i_a \times i_b) \qquad \begin{array}{ll} \text{nếu } i_vld: & acc \leftarrow sum \\ \text{nếu } i_vld \wedge i_last: & res \leftarrow sum \end{array}$$

Bảng 5.2: `gemm_pe`

ID	Yêu cầu
PE-01	Phép nhân là INT8 × INT8 có dấu cho tích 16 bit có dấu, mở rộng dấu lên 32 bit; phép cộng 32 bit wrap mod 2^{32}. Toàn bộ nhân-cộng nằm trong một chu kỳ (tổ hợp từ `i_a`, `i_b`, `acc` tới thanh ghi)
PE-02	`acc` và `res` chỉ thay đổi khi `i_vld` = 1 (và `i_last` = 1 với `res`). `i_first` = 1 nạp tích mới, không cộng với `acc` cũ; không có chu kỳ xóa
PE-03	`o_a`, `o_b` cập nhật mọi chu kỳ (được phép chặn khi `i_vld` = 0 để tiết kiệm năng lượng). `o_vld`, `o_first`, `o_last` cập nhật mọi chu kỳ
PE-04	`rst_n` = 0 hoặc `i_clr` = 1: `o_vld`, `o_first`, `o_last` về 0. `acc`, `res`, `o_a`, `o_b` không bắt buộc reset
PE-05	`o_res` nối thẳng từ thanh ghi `res`

```
// Tham khảo (SystemVerilog)
logic signed [15:0] prod;
logic signed [31:0] sum;
assign prod = $signed(i_a) * $signed(i_b);
assign sum  = (i_first ? 32'sd0 : acc) + 32'(prod);
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n)       {o_vld, o_first, o_last} <= '0;
  else if (i_clr)   {o_vld, o_first, o_last} <= '0;
  else              {o_vld, o_first, o_last} <= {i_vld, i_first, i_last};
always_ff @(posedge clk) begin
  o_a <= i_a;  o_b <= i_b;
  if (i_vld)           acc <= sum;
  if (i_vld && i_last) res <= sum;
end
```

5.2 gemm_tile_seq

Bảng 5.3: Port của gemm_tile_seq

Port	Rộng	Chiều	Mô tả
clk, rst_n, i_clr	1	In	
i_start	1	In	Nối i_pe_start
i_matrix_dim	24	In	[7:0] M, [15:8] N, [23:16] K
o_rd_en	1	Out	Nối o_a_rd_en và o_b_rd_en
o_rd_i0, o_rd_j0, o_rd_k	5 mỗi port	Out	Nối o_a_rd_row, o_b_rd_col, o_a_rd_k = o_b_rd_k
o_row_vld	P	Out	Bit r = 1 khi $i_0 + r < M$ (chỉ có nghĩa khi o_rd_en = 1)
o_first, o_last	1, 1	Out	k = 0, k = K−1 (chỉ có nghĩa khi o_rd_en = 1)
o_meta_push	1	Out	= o_rd_en ∧ (k = K−1): đẩy metadata của tile đang phát
o_meta_i0, o_meta_j0	5, 5	Out	i_0, j_0 của tile
o_meta_mask	P	Out	Bit c = 1 khi $j_0 + c < N$
o_meta_rows	$\lceil \log_2(P+1) \rceil$	Out	Số hàng hợp lệ = min(P, M − i_0), 1 … P
o_meta_last_tile	1	Out	Tile đang phát là tile cuối
o_issuing	1	Out	Đang trong pha phát (trạng thái ISSUE)

Bảng 5.4: gemm_tile_seq

ID	Yêu cầu
SEQ-01	Hai trạng thái: IDLE và ISSUE. i_start ở IDLE: chốt M, N, K; tính T_M, T_N, $N_T = T_M T_N$ và $I = \max(K, P)$; đặt tile n = 0, (t_i, t_j) = (0, 0), bộ đếm chu kỳ trong tile cnt = 0; sang ISSUE
SEQ-02	Trong ISSUE, mỗi chu kỳ là một "khe" cnt = 0 … I−1 của tile n. Khe cnt < K phát yêu cầu đọc k = cnt; khe cnt ≥ K là bọt (o_rd_en = 0). Sau khe I−1: sang tile kế tiếp theo thứ tự t_j chạy trong, t_i chạy ngoài; sau tile $N_T - 1$: về IDLE
SEQ-03	Mọi output là hàm của thanh ghi trạng thái (không phụ thuộc tổ hợp vào input trong cùng chu kỳ). $i_0 = t_i P$, $j_0 = t_j P$
SEQ-04	Hệ quả: yêu cầu đọc k của tile n nằm ở chu kỳ $S_n + k$, với $S_n = 1 + n \cdot I$

5.3 gemm_skew

Bảng 5.5: Port của gemm_skew

Port	Rộng	Chiều	Mô tả
clk, rst_n, i_clr	1	In	
i_rd_en, i_row_vld, i_first, i_last	1, P, 1, 1	In	Từ gemm_tile_seq, cùng chu kỳ với yêu cầu đọc
i_a_data, i_b_data	8P, 8P	In	Từ SRAM, chu kỳ sau yêu cầu
o_west_a	8P	Out	Lane r → i_a của PE(r, 0)
o_west_vld, o_west_first, o_west_last	P mỗi port	Out	Bit r → i_vld, i_first, i_last của PE(r, 0)
o_north_b	8P	Out	Lane c → i_b của PE(0, c)

Bảng 5.6: gemm_skew

ID	Yêu cầu
SKW-01	Tầng căn chỉnh: một thanh ghi chốt {i_rd_en ∧ i_row_vld[r], i_first, i_last}, cho cờ của mỗi hàng r ở chu kỳ sau yêu cầu, cùng lúc với dữ liệu SRAM. Thanh ghi này reset/clear về 0
SKW-02	Lane r của A cùng ba cờ của hàng r đi qua đúng r thanh ghi nối tiếp; lane 0 không qua thanh ghi nào. Thanh ghi cờ reset/clear về 0
SKW-03	Lane c của B đi qua đúng c thanh ghi nối tiếp; lane 0 không qua thanh ghi nào
SKW-04	Tổng số tầng: $P(P-1)/2$ cho A (mỗi tầng 8 + 3 bit) và $P(P-1)/2$ cho B (mỗi tầng 8 bit)

5.4 gemm_pe_grid

Bảng 5.7: gemm_pe_grid

ID	Yêu cầu
GRD-01	PE(r, c), $0 \le$ r, c $<$ P. Đầu vào A và cờ của PE(r, 0) lấy từ o_west_* lane r; của PE(r, c $>$ 0) lấy từ o_a, o_vld, o_first, o_last của PE(r, c−1)
GRD-02	i_b của PE(0, c) lấy từ o_north_b lane c; của PE(r $>$ 0, c) lấy từ o_b của PE(r−1, c)
GRD-03	Output của lưới: o_res của mọi PE, xếp res_bus[(r·P+c)·32+:32]; o_vld và o_last của PE(r, P−1) cho mỗi r (east_vld[r], east_last[r])
GRD-04	o_a của cột cuối và o_b của hàng cuối không nối đi đâu (công cụ tổng hợp tự bỏ)

5.5 gemm_drain

Bảng 5.8: Port của gemm_drain

Port	Rộng	Chiều	Mô tả
clk, rst_n, i_clr	1	In	
i_res_bus	$32P^2$	In	res của mọi PE
i_east_vld, i_east_last	P, P	In	o_vld, o_last của PE(r, P−1)
i_meta_*	—	In	Từ gemm_tile_seq: push, i0, j0, mask, rows, last_tile
o_row_*	như bảng 3.1	Out	Luồng hàng sang VPU
o_final	1	Out	= 1 cùng chu kỳ với o_row_valid của đoạn hàng cuối cùng

Bảng 5.9: `gemm_drain`

ID	Yêu cầu
DRN-01	Tín hiệu kích hoạt của hàng r: `trig[r]=i_east_vld[r]∧i_east_last[r]`. Nó bằng 1 đúng ở chu kỳ đầu tiên mà `res` của cả hàng r đã chứa kết quả của tile vừa xong. Tại mỗi chu kỳ có tối đa một bit `trig` bằng 1
DRN-02	FIFO metadata sâu 2 mục: đẩy khi `i_meta_push`, lấy ra khi `trig[0]` = 1. Mục vừa lấy ra trở thành "tile đang xả" và được giữ tới lần `trig[0]` kế tiếp
DRN-03	Khi `trig[r]` = 1 (với r = 0 dùng mục vừa lấy ra): ở cạnh clock cuối chu kỳ đó, thanh ghi đầu ra nạp `o_row_valid` = 1, `o_row_idx` = $i_0 + r$, `o_row_col` = j_0, `o_row_data` = `res` của hàng r, `o_row_mask` = mask của tile, `o_final` = (`last_tile` ∧ r = `rows` − 1). Không có `trig`: `o_row_valid` = `o_final` = 0
DRN-04	Hàng không hợp lệ ($i_0 + r \geq M$) có `vld` = 0 trong mọi PE của nó, nên `trig[r]` không bao giờ bật cho hàng đó: không cần lọc thêm
DRN-05	Mux chọn hàng là P:1 trên P×32 bit, điều khiển bởi `trig` (one-hot)

5.6 o_busy và o_pe_done

Bảng 5.10: Logic trong top

ID	Yêu cầu
TOP-01	`o_pe_done` = `o_final` trễ đúng `L_VPU` + 1 chu kỳ (đường trễ thanh ghi, clear về 0)
TOP-02	`o_busy`: đặt 1 ở cạnh clock cuối chu kỳ có `i_pe_start` (khi đang rảnh), về 0 ở cạnh clock cuối chu kỳ có `o_pe_done`

6 Timing

Mọi sự kiện có thời điểm cố định, tính được từ M, N, K, P và `L_VPU`. Các công thức dưới đây là quy định: RTL phải khớp đúng từng chu kỳ (kiểm bằng `gemm_systolic_array_model.py`).

Bảng 6.1: Thời điểm các sự kiện (chu kỳ 0 = chu kỳ có `i_pe_start`)

Sự kiện	Chu kỳ
Khoảng cách giữa hai tile	$I = \max(K, P)$
Yêu cầu đọc k của tile n	$S_n + k$, với $S_n = 1 + nI$
Dữ liệu SRAM của yêu cầu đó	$S_n + k + 1$
PE(r, c) nhận phần tử k của tile n	$S_n + 1 + k + r + c$
`res` của PE(r, c) chứa tile n từ	$S_n + 1 + K + r + c$
`trig[r]` của tile n	$S_n + K + P + r$
`o_row_valid` của hàng r, tile n	$D_n(r) = S_n + K + P + 1 + r$
`o_pe_done`	$D_{last} + L_{VPU} + 1 = (N_T - 1)I + K + P + r_{last} + L_{VPU} + 3$

Trong đó $N_T = \lceil M/P \rceil \lceil N/P \rceil$ và $r_{last} = \min(P, M - (T_M - 1)P) - 1$ là chỉ số hàng hợp lệ cuối của tile cuối.

VÌ SAO I = MAX(K, P) LÀ ĐỦ

- $I \geq K$: mỗi tile cần K lần đọc liên tiếp trên một cổng.
- $I \geq P$: `res` của PE(r, 0) bị tile n+1 ghi đè ở cạnh clock cuối chu kỳ $S_n + I + K + r$, không sớm hơn chu kỳ $S_n + K + P + r$ mà khối xả đọc hàng r của tile n. Thanh ghi đọc trước khi bị ghi trong cùng cạnh clock, nên giá trị xả là của tile n.
- $I \geq P$ cũng làm P đoạn hàng của một tile và của tile sau không bao giờ trùng chu kỳ, nên cổng xả chỉ cần một

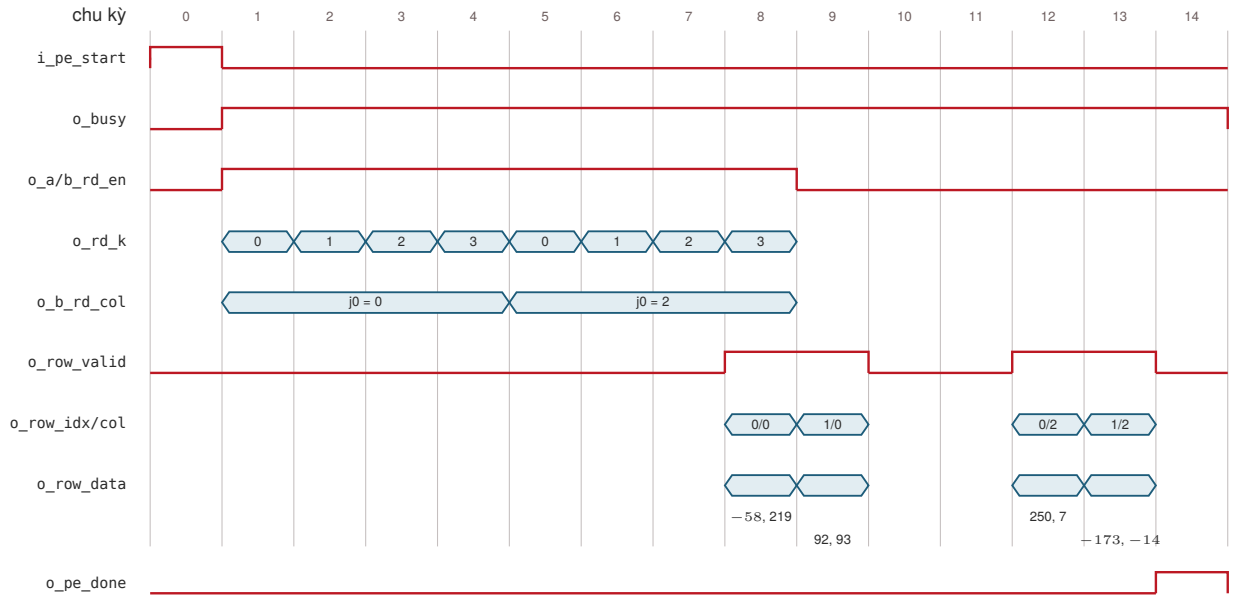
đoạn hàng mỗi chu kỳ và FIFO metadata chỉ cần 2 mục.

Bảng 6.2: Số chu kỳ từ `i_pe_start` tới `o_pe_done`

(M, N, K)	P = 4, L = 0	P = 8, L = 0	P = 4, L = 4	P = 8, L = 4
(32, 32, 32)	2058	530	2062	534
(20, 12, 8)	130	62	134	66
(1, 4, 4)	11	15	15	19
(2, 4, 4), P = 2 (ví dụ minh họa)	14			

6.1 Dạng sóng minh họa

Ví dụ M = 2, N = 4, K = 4 với P = 2, `L_VPU` = 0 (cùng ví dụ của tài liệu thuật toán): 2 tile, I = 4.



Hình 6.1: Dạng sóng của ví dụ: tile 0 đọc ở chu kỳ 1–4, tile 1 ở chu kỳ 5–8; hàng xả ở chu kỳ 8, 9, 12, 13; `o_pe_done` ở chu kỳ 14

7 Reset và soft clear

Bảng 7.1: Reset

ID	Yêu cầu
RST-01	`rst_n` = 0 (bất đồng bộ): `gemm_tile_seq` về IDLE; mọi thanh ghi cờ (`vld`, `first`, `last` trong `gemm_skew` và `gemm_pe`), FIFO metadata, `o_row_valid`, `o_final`, đường trễ `o_pe_done`, `o_busy` về 0
RST-02	`i_soft_clr` = 1 (đồng bộ): tác động như RST-01 ở cạnh clock cuối chu kỳ đó. Từ chu kỳ kế tiếp array rảnh; không có `o_pe_done` cho lượt bị hủy, không có `o_row_valid` nào của lượt đó
RST-03	Thanh ghi dữ liệu (`acc`, `res`, `a`, `b` trong PE và tầng làm lệch, dữ liệu đầu ra) không bắt buộc reset; mọi đường dùng chúng đều bị chặn bởi cờ hợp lệ đã được reset
RST-04	`i_soft_clr` và `i_pe_start` không bao giờ cùng chu kỳ (FSM bảo đảm); nếu có, `i_soft_clr` thắng

8 Quy tắc hiện thực

Bảng 8.1: Hiện thực

ID	Yêu cầu
IMP-01	SystemVerilog tổng hợp được; `always_ff@(posedgeclkornegedgerst_n)` cho thanh ghi có reset, `always_ff@(posedgeclk)` cho thanh ghi dữ liệu; không có latch
IMP-02	Mọi phép nhân, cộng dùng kiểu `signed` rõ ràng (`$signed`); không dùng phép chia hay modulo: $\lceil M/P \rceil$ tính bằng dịch vì P là lũy thừa của 2
IMP-03	Mỗi module con trong file riêng, cùng tên module; lưới PE và các tầng làm lệch dựng bằng `generate`
IMP-04	Không có đường tổ hợp từ input tới output của `gemm_systolic_array`: các port `o_*` đều lấy từ thanh ghi

Bảng 8.2: Assertion bắt buộc trong RTL (SVA, chỉ trong mô phỏng)

ID	Yêu cầu
AS-01	`i_pe_start` $\rightarrow$ `!o_busy`
AS-02	`o_a_rd_en==o_b_rd_en`
AS-03	`$onehot0(trig)` trong `gemm_drain`
AS-04	FIFO metadata không bao giờ đẩy khi đầy, không bao giờ lấy khi rỗng
AS-05	`o_busy` $\rightarrow$ `$stable(i_matrix_dim)`
AS-06	`o_row_valid` $\rightarrow$ (`o_row_idx` < M) $\wedge$ (`o_row_col` < N)
AS-07	Mỗi `i_pe_start` có đúng một `o_pe_done` trước `i_pe_start` kế tiếp

Bảng 8.3: Tài nguyên ước tính

Hạng mục	P = 4	P = 8
Bộ nhân 8 $\times$ 8 có dấu	16	64
Bộ cộng 32 bit	16	64
Thanh ghi trong lưới PE (83 bit mỗi PE)	1 328 bit	5 312 bit
Thanh ghi làm lệch (A 11 bit, B 8 bit mỗi tầng)	6 tầng: 114 bit	28 tầng: 532 bit
Mux xả	4:1 $\times$ 128 bit	8:1 $\times$ 256 bit